Programing Task

---

1)

At first, we have to define the HLA dataset and to load it:(when we want to train the model we want the input to be a 180-dimensional vector and to do that we want the data to be in a 180-dimensional vector so we have to convert it, to do that we have to now the 20 type of protein, when searching in google the types is: "ACDEFGHIKLMNPQRSTVWY")

so now to do instanse of the data and split it for test loadre and train loader:

2)a

INPUT

I would represent the 9-mers amino acids by one hot encoding ,each amino acid in the 9-mers amino acid is a 20-dimensional vecotr (because we have 20 types of standart amino acid) ,and the 9-mers amino acids is these vectors flattened for one vector so the input is a vector of 180 number/neuroun.

The reason behind this is that there is no order in the types of the amino acids no one of them is bigger of lower ... its not a numerical data. so the one hot encoding ensures that each amino acid is equally distinct thats mean all the types of the amino acid is equals no one is bigger or smaller...

OUTPUT

the output is a 7-dimensional vecotr of raw numbers (not proballity) which corresponds to the 7 classes : the class 0 is the negative one and the 1-6 is the alleles

The reason behind this is that we are asked to discover if a peptide was detected or not from one of the 6 alleles and if so we have to discover which allele detected it. This is a classification problem so we want to use Cross Entropy loss function and in pyTorch we use nn.CrossEntropyLoss and so on this function applies at the biggining the softmax and then calculate the loss thats why we uesd a raw numbers

2)b

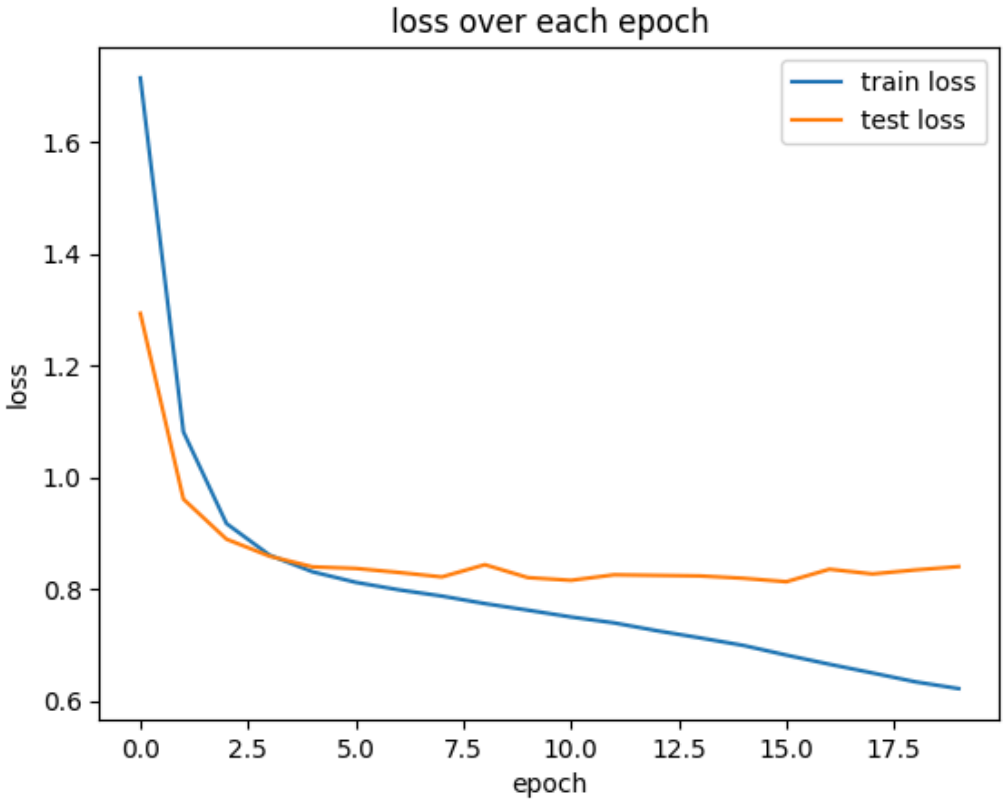
According to what we discussed above the input dimention is 180 neurouns (9 positions in the 9-mers amino acid and each position is a 20-dimensional vector, 9*20=180)

train, test the model and plot the results:

define the MLP:

```
Start coding or generate with AI.
```

```
100%                                                  20/20 [01:03<00:00,  3.07s/it]
Epoch: [1/20] | Training loss: 1.7142 | Test loss: 1.2930
Epoch: [2/20] | Training loss: 1.0813 | Test loss: 0.9610
Epoch: [3/20] | Training loss: 0.9172 | Test loss: 0.8894
Epoch: [4/20] | Training loss: 0.8608 | Test loss: 0.8589
Epoch: [5/20] | Training loss: 0.8310 | Test loss: 0.8398
Epoch: [6/20] | Training loss: 0.8121 | Test loss: 0.8370
Epoch: [7/20] | Training loss: 0.7988 | Test loss: 0.8298
Epoch: [8/20] | Training loss: 0.7876 | Test loss: 0.8218
Epoch: [9/20] | Training loss: 0.7741 | Test loss: 0.8436
Epoch: [10/20] | Training loss: 0.7623 | Test loss: 0.8205
Epoch: [11/20] | Training loss: 0.7501 | Test loss: 0.8157
Epoch: [12/20] | Training loss: 0.7396 | Test loss: 0.8256
Epoch: [13/20] | Training loss: 0.7255 | Test loss: 0.8246
Epoch: [14/20] | Training loss: 0.7127 | Test loss: 0.8236
Epoch: [15/20] | Training loss: 0.6993 | Test loss: 0.8194
Epoch: [16/20] | Training loss: 0.6820 | Test loss: 0.8132
Epoch: [17/20] | Training loss: 0.6656 | Test loss: 0.8356
Epoch: [18/20] | Training loss: 0.6501 | Test loss: 0.8271
Epoch: [19/20] | Training loss: 0.6341 | Test loss: 0.8343
Epoch: [20/20] | Training loss: 0.6221 | Test loss: 0.8402
```



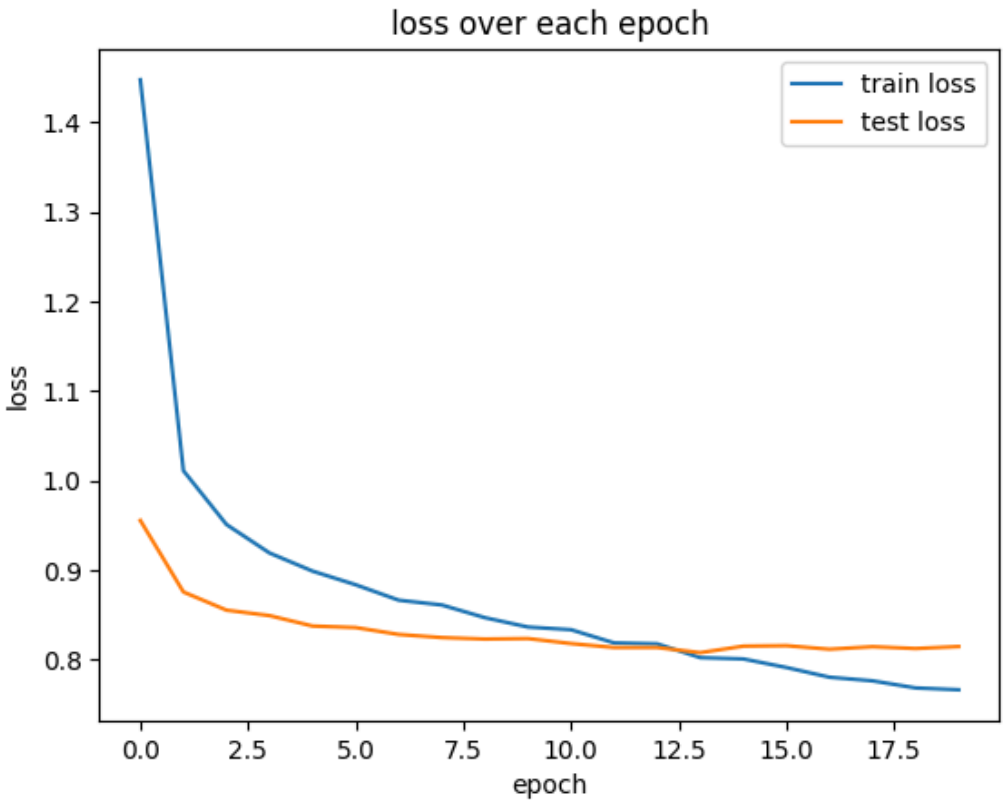
We can see that these dimensions is posing a problem: the test loss is getting stuck at some point that even when we train the network more (more epochs) the test loss is not decreases and that is a problem because we expect that network will get "closer" to the right answer when prediction.

2)c

When searching about the problem, the problem is on the training and its called **Overfitting**.
the solution is to lower the neurons in the first hidden layer and then we have to prevent **Overfitting** by using Droput, so our new network is : the input layer is 180-dimension then we 2 hidden layers the first one is 64 neurons then 32 then the 7- dimension output layer so the new network is : so our new architecture is 180 -> 90 -> 45 -> 7

```
Start coding or generate with AI.
```

```
100%                                                  20/20 [01:00<00:00,  2.94s/it]
Epoch: [1/20] | Training loss: 1.4470 | Test loss: 0.9556
Epoch: [2/20] | Training loss: 1.0111 | Test loss: 0.8758
Epoch: [3/20] | Training loss: 0.9511 | Test loss: 0.8555
Epoch: [4/20] | Training loss: 0.9193 | Test loss: 0.8495
Epoch: [5/20] | Training loss: 0.8991 | Test loss: 0.8378
Epoch: [6/20] | Training loss: 0.8839 | Test loss: 0.8361
Epoch: [7/20] | Training loss: 0.8666 | Test loss: 0.8284
Epoch: [8/20] | Training loss: 0.8613 | Test loss: 0.8250
Epoch: [9/20] | Training loss: 0.8472 | Test loss: 0.8233
Epoch: [10/20] | Training loss: 0.8367 | Test loss: 0.8238
Epoch: [11/20] | Training loss: 0.8337 | Test loss: 0.8183
Epoch: [12/20] | Training loss: 0.8190 | Test loss: 0.8139
Epoch: [13/20] | Training loss: 0.8179 | Test loss: 0.8139
Epoch: [14/20] | Training loss: 0.8026 | Test loss: 0.8081
Epoch: [15/20] | Training loss: 0.8011 | Test loss: 0.8154
Epoch: [16/20] | Training loss: 0.7916 | Test loss: 0.8159
Epoch: [17/20] | Training loss: 0.7808 | Test loss: 0.8120
Epoch: [18/20] | Training loss: 0.7767 | Test loss: 0.8148
Epoch: [19/20] | Training loss: 0.7688 | Test loss: 0.8128
Epoch: [20/20] | Training loss: 0.7667 | Test loss: 0.8149
```

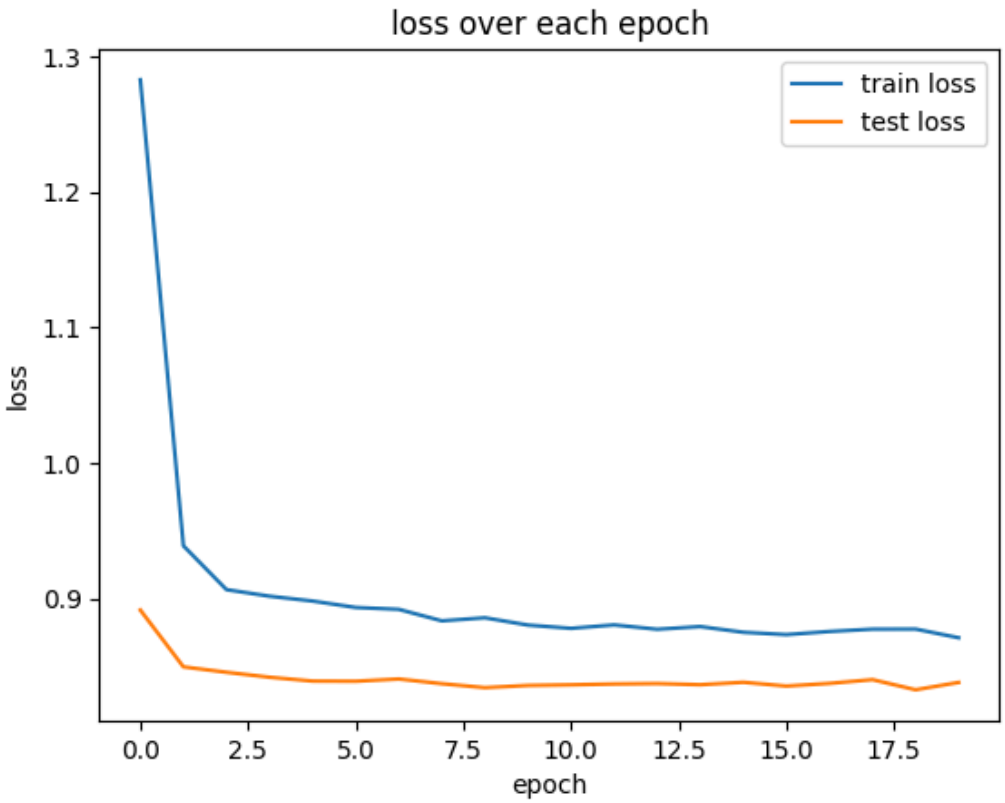


2)d

We have to remove the activation functions from the network(the relu between the layers) so we will get :

```
Start coding or generate with AI.
```

```
100%                                                  20/20 [01:03<00:00,  3.20s/it]
Epoch: [1/20] | Training loss: 1.2829 | Test loss: 0.8913
Epoch: [2/20] | Training loss: 0.9388 | Test loss: 0.8494
Epoch: [3/20] | Training loss: 0.9064 | Test loss: 0.8454
Epoch: [4/20] | Training loss: 0.9015 | Test loss: 0.8417
Epoch: [5/20] | Training loss: 0.8980 | Test loss: 0.8389
Epoch: [6/20] | Training loss: 0.8932 | Test loss: 0.8387
Epoch: [7/20] | Training loss: 0.8918 | Test loss: 0.8404
Epoch: [8/20] | Training loss: 0.8833 | Test loss: 0.8369
Epoch: [9/20] | Training loss: 0.8856 | Test loss: 0.8340
Epoch: [10/20] | Training loss: 0.8802 | Test loss: 0.8356
Epoch: [11/20] | Training loss: 0.8778 | Test loss: 0.8361
Epoch: [12/20] | Training loss: 0.8803 | Test loss: 0.8367
Epoch: [13/20] | Training loss: 0.8771 | Test loss: 0.8370
Epoch: [14/20] | Training loss: 0.8791 | Test loss: 0.8362
Epoch: [15/20] | Training loss: 0.8749 | Test loss: 0.8380
Epoch: [16/20] | Training loss: 0.8732 | Test loss: 0.8352
Epoch: [17/20] | Training loss: 0.8755 | Test loss: 0.8372
Epoch: [18/20] | Training loss: 0.8772 | Test loss: 0.8399
Epoch: [19/20] | Training loss: 0.8773 | Test loss: 0.8324
Epoch: [20/20] | Training loss: 0.8709 | Test loss: 0.8379
```



2)e

first of all we have to download all the sequence then separate all its consecutive 9-mer segments, then use the model to predict:

```
Start coding or generate with AI.
```

```
Rank 1: STEIYQAGS | Allele: A0101 | Score: 0.9805
Rank 2: LTESNKKFL | Allele: A0101 | Score: 0.9670
Rank 3: ESEFRVYSS | Allele: A0101 | Score: 0.9628
```

Theoretical Questions

1)

Linear transforamtion is that satisfies additivity and scalar multiplication. suppose we have,$T_1 : V \to W$, $T_2 : W \to U$, now define $T = T_2 \circ T_1$, for every $v, w \in V$:

$T(v+w) = (T_1 \circ T_2)(v+w) = T_1(T_2(v+w)) = T_1(T_2(v) + T_2(w)) = T_1(T_2(v)) + T_1(T_2(w)) =$
$+ (T_1 \circ T_2)(w) = T(v) + T(w)$

for $a \in F$:
$T(av) = (T_1 \circ T_2)(av) = T_1(T_2(av)) = T_1(aT_2(v)) = aT_1(T_2(v)) = a(T_1 \circ T_2)(v) = aT(v)$

the transitions is done due to the linearity of T_1, T_2.
T is a composition of linear function and its also linear function so we prove what we want.

An affine transformation $T : V \to W$ is defined as the sum of a linear transformation and a constant translation vector :

$$T(v) = Av + w$$

so we define $T_1 : V \to W$,$T_2 : W \to U$, that is $T_1(v) = A_1 v + w_1, T_2(u) = A_2 u + w_2$ now define $T = T_2 \circ T_1$, for every $v \in V$:

$T(v) = (T_2 \circ T_1)(v) = T_2(T_1(v)) = T_2(A_1 v + w_1) = A_2(A_1 v + w_1) + w_2 = A_2 A_1 v + A_2 w_1 + w_2$
so we got a linear function $T_3(v) = A_2 A_1 v$ and $u = A_2 w_1 + w_2$ is a constant so we got that $T(v) = T_3(v) + u$ so also T is affine transformation and thats what we want.

2)

On circle when we want to go from vector to another by just rotating the vector, we have to go throw one of two diffrent way: the first one is just go throw the direct way ,with the clock arrows, and the second one is to go throw the oppisite way, the oppisite move of the clock arrows, so formally the loss have to be the min between:($\hat{y}$ is the prediction and y is the label)

- loss = |$\hat{y}$ - y|
- loss = 360-|$\hat{y}$ - y|
  and we have to return the squared of the loss to "punish" large mistake so the pseuso code is :

circular_loss($\hat{y}, y$) {
$deff$=$|\hat{y} - y|$
$loss$=$min\{deff$, $360 - deff\}$

return $loss^2$

}

3)

a)

Let the arguments be $u = x + y$, $v = 2x$, and $w = z$.Applying the chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial u}\frac{\partial u}{\partial x} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial x} + \frac{\partial f}{\partial w}\frac{\partial w}{\partial x}$$

so $\frac{\partial u}{\partial x} = 1, \frac{\partial v}{\partial x} = 2, \frac{\partial w}{\partial x} = 0$ (since z is independent of x) so we got

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial u} * 1 + \frac{\partial f}{\partial v} * 2$$

b)

This is the chain rule applied n times.Let $g_i = f_i(f_{i+1}(\dots f_n(x)))$.Result, so we know from the chain rule that

$$g_i' = f_i'(f_{i+1}(\dots f_n(x))) \cdot f_{i+1}'(f_{i+2}(\dots f_n(x))) \cdots f_{n-1}'(f_n(x)) \cdot f_n'(x)$$

formally we can prove it by inductoin:

- $\underline{i = n}$:
  Its true due to the chain rule.
- we suppose that its true for $i = k + 1$ and prove for $i = k$:
  $g_k = f_k(f_{k+1}(\dots f_n(x)))$ so applying the chain rule we get that
  $g_k' = f_k'(f_{k+1}(\dots f_n(x)))(f_{k+1}(\dots f_n(x)))' = f_k'(f_{k+1}(\dots f_n(x)))g_{k+1}' = f_k'(f_{k+1}(\dots f_n(x)))f \cdots f_{n-1}'(f_n(x)) \cdot f_n'(x)$

  and thats what we want (the last transition is done due to the I.H).

now pay attention that $g_1(x) = f_1(f_2(\dots f_n(x)))$ so we get that

$$g_1(x)' = (f_1(f_2(\dots f_n(x))))' = f_1'(f_2(\dots f_n(x))) \cdot f_2'(f_3(\dots f_n(x))) \cdots f_{n-1}'(f_n(x)) \cdot f_n'(x)$$

so the last result is :

$$f_1'(f_2(\dots f_n(x))) \cdot f_2'(f_3(\dots f_n(x))) \cdots f_{n-1}'(f_n(x)) \cdot f_n'(x)$$

c)

like the first section, we define for all $i \in [n-1]$:

$$g_i = f_i(x, f_{i+1}(x, f_{i+2}(\dots f_{n-1}(x, f_n(x))))) = f_i(x, g_{i+1})$$

when $g_n = f_n(x)$ so

$$g_i' = \frac{d}{dx} f_i(x, g_{i+1}) + \frac{d}{dg_{i+1}} f_i(x, g_{i+1}) \frac{d}{dx} g_{i+1}'$$

we want to calculate g_1' so the answer is :

$$g_1' = \frac{d}{dx} f_1(x, g_2) + \frac{d}{dg_2} f_1(x, g_2) \frac{d}{dx} g_2'$$

d)

Let's break this down from the outside in.Outer layer: f has one argument $u = x + g(x + h(x))$.Inner layer: g has one argument $v = x + h(x)$.Applying the chain rule:

$$\frac{d}{dx} f(u) = f'(u) \cdot \frac{du}{dx}$$
$$\frac{du}{dx} = \frac{d}{dx}(x) + \frac{d}{dx} g(v) = 1 + g'(v) \cdot \frac{dv}{dx}$$
$$\frac{dv}{dx} = \frac{d}{dx}(x) + \frac{d}{dx} h(x) = 1 + h'(x)$$

Result:

$$f'(x + g(x + h(x))) \cdot [1 + g'(x + h(x)) \cdot (1 + h'(x))]$$